

解题报告

姓名：白家辉

班级：计算机大类2508

学号：8208250831

日期：2026.5.8

总览

本次解题报告中，共完成 4 道题目。

题目名称	难度	知识点
P1598 垂直柱状图	普及-	计数统计、格式输出
P2239 螺旋矩阵	普及/提高-	分层模拟、数学规律
P4715 淘汰赛	普及-	模拟、递归思维
P4913 二叉树深度	普及-	二叉树、迭代遍历、深度统计

（一）垂直柱状图

题目信息

题目链接：<https://www.luogu.com.cn/problem/P1598>

知识点：计数统计、格式输出

题目简述

输入四行只包含大写字母的文本，每行长度不超过 100。需要统计 A 到 Z 这 26 个字母分别出现了多少次，并按照样例要求输出一幅竖直方向的柱状图。

输出时不仅要保证每一层的星号位置正确，还要特别注意行尾不能多输出空格。

1.00s, 128MB。

题目分析

1. 读题

这题的核心不是复杂算法，而是“先统计，再按固定格式打印”。真正容易出错的部分主要在输出细节，尤其是每行末尾多余空格的控制。

2. 性质观察

柱状图的高度只取决于出现次数最多的那个字母。因此可以先统计全部字母频次，再从最大高度往下逐层判断：当前这一层某个字母位置是否应该打印 *。

3. 程序设计思路

处理过程分成两步：

1. 先扫描四行字符串，统计 A 到 Z 每个字母的出现次数；
2. 找出最大出现次数 $\max n$ ；
3. 从高度 $\max n$ 这一层开始，一层一层向下输出；
4. 对于每层的每个字母，若它的出现次数不少于当前层高，就输出 *，否则输出空格；
5. 最后一行再输出字母 A 到 Z 作为横坐标。

当前代码中又进一步开了一个二维字符数组 `print` 来保存整张柱状图，然后统一按行输出。这种写法比边判断边打印更直观，也更容易控制整体格式。

4. 数据结构与算法选择

本题核心只需要一个长度为 26 的计数数组。当前实现额外使用了一个字符矩阵保存最终图形，因此整体做法可以概括为“计数 + 构图 + 输出”。

5. 时空复杂度分析

- 时间复杂度： $O(26 \times H + L)$ ，其中 H 为柱状图最大高度， L 为输入总字符数。由于规模很小，可视作常数级。
- 空间复杂度： $O(26)$ ，仅需要一个字母计数数组。

代码实现

洛谷 / 评测记录 / 评测详情

R281460912 记录详情

编程语言

C O2

代码长度

991B

用时

17ms

内存

824.00KB

测试点信息

源代码

测试点信息

#1

AC

3ms/788.00KB

#2

AC

3ms/788.00KB

#3

AC

4ms/824.00KB

#4

AC

3ms/788.00KB

#5

AC

4ms/792.00KB

jsj0708Baijiahui

所属题目

P1598 [USACO03FEB] 垂直柱状...

评测状态

Accepted

评测分数

100

提交时间

2026-06-11 17:31:27

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
int main()
{
    int arr[26] = { 0 };
    int n = 4;
    while (n--)
    {
        char s[100] = {0};
        fgets(s, sizeof(s), stdin);
        int sz = strlen(s);
        for (int i = 0; i < sz; i++)
        {
            int temp = s[i] - 'A';
            if (temp >= 0 && temp < 26)
            {
                arr[temp]++;
            }
        }
    }
    int max = 0;
    for (int i = 0; i < 26; i++)
    {
        if (arr[i] > max)
        {
            max = arr[i];
        }
    }
    char** print = (char**)malloc((max+1) * sizeof(char*));
    for (int i = 0; i <= max; i++)
    {
        print[i] = (char*)malloc(26 * sizeof(char));
    }
    for (int i = 0; i < 26; i++)
    {
        print[max][i] = i + 'A';
    }

    for (int i = 0; i < max; i++)
    {
        for (int j = 0; j < 26; j++)
        {

```

```

        if (arr[j] >= max - i)
        {
            print[i][j] = '*';
        }
        else
        {
            print[i][j] = ' ';
        }
    }
}
for (int i = 0; i <= max; i++)
{
    for (int j = 0; j < 26; j++)
    {
        printf("%c ", print[i][j]);
    }
    printf("\n");
}
for (int i = 0; i <= max; i++)
{
    free(print[i]);
}
free(print);
}

```

(二) 螺旋矩阵

题目信息

题目链接：<https://www.luogu.com.cn/problem/P2239>

知识点：分层模拟、数学规律

题目简述

按顺时针螺旋方式在一个 $n \times n$ 矩阵中依次填入 $1, 2, 3, \dots, n^2$ 。给定矩阵大小 n 以及位置 (i, j) ，要求求出该位置上的数字。

对于全部数据， n 最大可达到 30000，因此不能真的把整个矩阵完整构造出来。

1.00s, 128MB。

题目分析

1. 读题

如果直接把整张矩阵存下来显然不可行，但题目只要求某一个位置的值，因此即使采用模拟，也应该做到“不建矩阵、只沿螺旋路线走到目标位置为止”。

2. 性质观察

螺旋矩阵可以看成一层一层的“方环”，而当前代码正是利用这一点，通过 `left`、`right`、`up`、`down` 四个边界来逐层缩小搜索范围。每走到螺旋路径上的一个位置，就把当前编号 `num` 加一，并检查是否命中了目标坐标。

3. 程序设计思路

当前实现采用逐层模拟：

1. 先把目标坐标转成从 0 开始计数的形式；
2. 维护当前方环的四条边界 `left`、`right`、`up`、`down`；
3. 按“上边从左到右、右边从上到下、下边从右到左、左边从下到上”的顺序依次走；
4. 每走到一个格子，就判断它是否为目标位置；
5. 一旦命中，就立即输出当前编号并结束。

这种写法虽然仍然是在模拟螺旋，但并没有真正开二维矩阵保存全部元素。

4. 数据结构与算法选择

本题当前实现没有额外数据结构，只用常数个边界变量和计数器完成逐层模拟。它的优势是空间占用极小，代码也比较直观。

5. 时空复杂度分析

- 时间复杂度：最坏情况下为 $O(n^2)$ ，因为当前代码可能沿着螺旋路径一路模拟到目标位置。
- 空间复杂度： $O(1)$ ，不需要存储矩阵。

代码实现

洛谷 / 评测记录 / 评测详情

R281460972 记录详情

编程语言C++2代码长度1.22KB用时1.59s内存1.08MB

测试点信息源代码

测试点信息

#1AC4ms/1.04MB	#2AC3ms/812.00KB	#3AC4ms/836.00KB	#4AC3ms/1.08MB	#5AC4ms/820.00KB	#6AC306ms/788.00KB	#7AC302ms/812.00KB
#8AC330ms/816.00KB	#9AC324ms/816.00KB	#10AC307ms/812.00KB				

jsj0708Baijiahui

所属题目P2239 [NOIP 2014 普及组] 螺旋...

评测状态Accepted

评测分数100

提交时间2026-06-11 17:31:58

```

#include<stdio.h>

int main()
{
    int N, temp1, temp2;
    scanf("%d %d", &N, &temp1);
    scanf("%d", &temp2);
    int target_x = temp1 - 1;
    int target_y = temp2 - 1;

    int left = 0, right = N - 1;
    int up = 0, down = N - 1;
    int num = 1;

    while (left <= right)
    {
        for (int y = left; y <= right; y++)
        {
            if (up == target_x && y == target_y)
            {
                printf("%d\n", num);
                return 0;
            }
            num++;
        }
        up++;
        for (int x = up; x <= down; x++)
        {
            if (x == target_x && right == target_y)
            {
                printf("%d\n", num);
                return 0;
            }
            num++;
        }
        right--;

        for (int y = right; y >= left; y--) {
            if (down == target_x && y == target_y) {
                printf("%d\n", num);
                return 0;
            }
        }
    }
}

```



```

        num++;
    }
    down--;

    for (int x = down; x >= up; x--) {
        if (x == target_x && left == target_y) {
            printf("%d\n", num);
            return 0;
        }
        num++;
    }
    left++;
}

return 0;
}

```

(三) 淘汰赛

题目信息

题目链接：<https://www.luogu.com.cn/problem/P4715>

知识点：模拟、递归思维

题目简述

有 2^n 个国家参加淘汰赛，每个国家有一个互不相同的能力值。比赛按固定对阵顺序进行，能力值更高的国家一定获胜。

题目要求输出亚军的编号，也就是最后一场决赛中失败的那个国家编号。

对于全部数据， $n \leq 7$ 。

1.00s, 128MB。

题目分析

1. 读题

因为强者一定能赢弱者，所以整个淘汰赛结果是完全确定的。题目并不是要找冠军，而是要找决赛输掉的那个国家，也就是“另一个进入决赛的人”。

2. 性质观察

整张赛程表可以视为左右两半分别决出一个冠军，最后这两个半区冠军再打一场决赛。因此亚军一定是左右两个半区冠军中能力值较小的那个。

这意味着我们并不需要记录每一场比赛的全部细节，只要分别找出左右半区的最强者即可。

3. 程序设计思路

当前实现采用的是逐轮模拟淘汰赛：

1. 第一轮先按输入顺序两两比较，把每组较强者压入数组 `arr`；
2. 同时把原始能力值保存在 `arr2` 中，便于最后反查编号；
3. 之后继续对 `arr` 中的晋级者两两比较，直到只剩下决赛的两个人；
4. 这两个人中能力值较小者就是亚军；
5. 最后再回到原数组里查出这位亚军对应的编号并输出。

4. 数据结构与算法选择

本题只需要数组存储每轮晋级者与原始能力值。当前代码选择了显式地一轮一轮模拟比赛过程，因此非常贴近题目描述，也容易看出“决赛两人是谁”。

5. 时空复杂度分析

- 时间复杂度： $O(2^n)$ ，只需要扫描一遍全部选手。
- 空间复杂度： $O(2^n)$ 或 $O(1)$ 额外空间，取决于具体实现方式。

由于 $n \leq 7$ ，总人数最多只有 128，任何线性做法都足够快。

代码实现

首页 / 评测记录 / 评测详情

R281461100 记录详情

编程语言

C O2

代码长度

1.11KB

用时

32ms

内存

1.04MB

测试点信息

源代码

测试点信息

#1

AC

3ms/812.00KB

#2

AC

3ms/640.00KB

#3

AC

3ms/1.04MB

#4

AC

3ms/1.04MB

#5

AC

3ms/816.00KB

#6

AC

4ms/816.00KB

#7

AC

3ms/812.00KB

#8

AC

3ms/788.00KB

#9

AC

4ms/812.00KB

#10

AC

3ms/1.04MB

jsj0708Baijiahui

所属题目

P4715 【深基16.例1】淘汰赛

评测状态

Accepted

评测分数

100

提交时间

2026-06-11 17:32:56

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
int main()
{
    int n = 0;
    int num = 1;
    int pos = 0;
    int pos2 = 0;
    scanf("%d", &n);
    if (n == 1)
    {
        int temp1 = 0;
        int temp2 = 0;
        scanf("%d %d", &temp1, &temp2);
        printf("%d\n", temp1 < temp2 ? 1 : 2);
        return 0;
    }
    for (int i = 0; i < n; i++)
    {
        num *= 2;
    }
    int* arr = (int*)malloc((num + 1) * sizeof(int));
    int* arr2 = (int*)malloc((num + 1) * sizeof(int));
    for (int i = 0; i < num; i += 2)
    {
        int temp1;
        int temp2;
        scanf("%d %d", &temp1, &temp2);
        arr2[pos2++] = temp1;
        arr2[pos2++] = temp2;
        if (temp1 > temp2)
            arr[pos++] = temp1;
        else
            arr[pos++] = temp2;
    }
    for (int i = 0; i < n - 2; i++)
    {
        int countrynum = pos;
        pos = 0;
        for (int j = 0; j < countrynum; j += 2)
        {
            int temp1 = arr[j];

```

```

        int temp2 = arr[j + 1];
        if (temp1 > temp2)
            arr[pos++] = temp1;
        else
            arr[pos++] = temp2;
    }
}
int temp1 = arr[0];
int temp2 = arr[1];
int result = temp1 < temp2 ? temp1 : temp2;
for (int i = 0; i < num; i++)
{
    if (arr2[i] == result)
    {
        printf("%d\n", i + 1);
        break;
    }
}
free(arr);
free(arr2);
return 0;
}

```

(四) 二叉树深度

题目信息

题目链接：<https://www.luogu.com.cn/problem/P4913>

知识点：二叉树、迭代遍历、深度统计

题目简述

给出一棵共有 n 个结点的二叉树，根结点编号固定为 1 。对于每个结点，输入其左儿子和右儿子的编号；若某个结点是叶子，则输入 $0\ 0$ 。

需要在整棵树建立完成后，求出这棵二叉树的深度，也就是从根到叶子最多经过的层数。

对于全部数据， $n \leq 10^6$ 。

1.00s, 128MB。

题目分析

1. 读题

题目输入的其实是一棵已经确定结构的二叉树，我们要做的只是从根结点出发，找到最深的那一层。由于每个结点的左右孩子编号都已经直接给出，因此建树本身并不复杂。

2. 性质观察

二叉树深度本质上就是“从根出发遍历整棵树时，遇到的最大层数”。无论采用深度优先还是广度优先，都能得到答案。

但本题的结点数可达到 10^6 。如果树退化成一条长链，递归 DFS 可能产生很深的调用栈，不够稳妥，因此更适合使用队列 BFS 或显式栈进行迭代遍历。

3. 程序设计思路

可以先用两个数组分别存每个结点的左儿子和右儿子，然后从根结点 1 开始做层次遍历：

1. 根结点入队，并记录当前层数为 1；
2. 每次取出一个结点时，查看它的左右儿子；
3. 若左儿子存在，就把它与“当前层数加一”一起入队；
4. 若右儿子存在，也做同样处理；
5. 在遍历过程中不断更新遇到的最大层数；
6. 队列清空后，最大层数就是整棵树的深度。

4. 数据结构与算法选择

本题适合使用数组存树，再配合队列进行广度优先遍历。数组便于按编号直接访问孩子结点，BFS 又能自然按层推进，并避免递归造成的栈深问题。

5. 时空复杂度分析

- 时间复杂度： $O(n)$ ，每个结点最多只会入队、出队各一次。
- 空间复杂度： $O(n)$ ，需要存储整棵树的左右儿子信息，遍历时队列最坏也可能达到线性规模。

在 10^6 级别数据下，线性遍历是合理且必要的做法。

代码实现

首页 / 评测记录 / 评测详情

R281461155 记录详情

编程语言

C++2

代码长度

740B

用时

44ms

内存

1.21MB

测试点信息

源代码

测试点信息

#1

AC

11ms/1008.00KB

#2

AC

6ms/788.00KB

#3

AC

9ms/1012.00KB

#4

AC

8ms/808.00KB

#5

AC

10ms/1.21MB

jsj0708Baijiahui

所属题目

P4913 【深基16.例3】二叉树深度

评测状态

Accepted

评测分数

100

提交时间

2026-06-11 17:33:20

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
int main()
{
    int n = 0;
    scanf("%d", &n);
    int* left = (int*)calloc((n+1) , sizeof(int));
    int* right = (int*)calloc((n+1) , sizeof(int));
    for (int i = 1; i <= n; i++)
    {
        scanf("%d %d", &left[i], &right[i]);
    }
    int* queue = (int*)malloc((n+1) * sizeof(int));
    queue[0] = 1;
    int front = 0;
    int rear = 1;
    int depth = 0;
    while (front < rear)
    {
        depth++;
        int size = rear - front;
        for (int i = 0; i < size; i++)
        {
            int temp = queue[front];
            if (left[temp] != 0)
            {
                queue[rear++] = left[temp];
            }
            if (right[temp] != 0)
            {
                queue[rear++] = right[temp];
            }
            front++;
        }
    }
    free(left);
    free(right);
    free(queue);
    printf("%d\n", depth);
    return 0;
}

```